I checked your uploaded Python file, Password Policy Validation Engine.py. I'll prepare the complete activity record based on your actual code, using simple language suitable for a B.Tech 1st-year CSE Data Science student.

Your program checks whether a password follows four classroom rules: minimum length of 8 characters, at least one uppercase letter, at least one lowercase letter, and at least one digit.

**Password Policy Validation Engine**

**1. Problem statement**

To develop a Python program that validates a password based on specified rules. The program checks the password length and verifies the presence of uppercase letters, lowercase letters, and digits. It displays appropriate error messages if any rule is not satisfied.

**2. Objectives**

1. To accept a sample password from the user.
2. To check whether the password contains at least 8 characters.
3. To check for at least one uppercase letter.
4. To check for at least one lowercase letter.
5. To check for at least one digit.
6. To display whether the password satisfies all the specified rules.

**3. Assumptions**

- The password is entered as a string.
- The minimum password length is 8 characters.
- At least one uppercase letter is required.
- At least one lowercase letter is required.
- At least one digit is required.
- Special characters are allowed but are not compulsory.
- The program checks only the specified classroom rules; it does not measure overall password security.

**4. Algorithm**

1. Start the program.
2. Input a sample password.
3. Set has_upper, has_lower, and has_digit to False.
4. Check each character in the password.
5. If a character is uppercase, set has_upper to True.
6. If a character is lowercase, set has_lower to True.
7. If a character is a digit, set has_digit to True.

8. Set valid to True.
9. Check whether the password has at least 8 characters. If not, display an error and set valid to False.
10. Check for an uppercase letter, lowercase letter, and digit. Display an error for every missing requirement.
11. If valid remains True, display that the password meets the classroom rules.
12. Otherwise, display that the password does not meet the rules.
13. Stop the program.

**5. Flowchart**

Start

Input password

Check every character for uppercase, lowercase, and digit

Check length and all three character requirements

Are all rules satisfied?

Yes

Display password meets rules

No

Display failed requirements and invalid result

End

Note: This is a simplified flowchart. The program checks each rule separately, so it can display more than one error message.

**6. Pseudocode**

```
BEGIN
  INPUT password

  has_upper = FALSE
  has_lower = FALSE
  has_digit = FALSE

  FOR each character IN password
    IF character is uppercase
      has_upper = TRUE
    END IF
```

```
    IF character is lowercase
        has_lower = TRUE
    END IF

    IF character is a digit
        has_digit = TRUE
    END IF
END FOR

valid = TRUE

IF length(password) < 8
    DISPLAY length error
    valid = FALSE
END IF

IF has_upper = FALSE
    DISPLAY uppercase error
    valid = FALSE
END IF

IF has_lower = FALSE
    DISPLAY lowercase error
    valid = FALSE
END IF

IF has_digit = FALSE
    DISPLAY digit error
    valid = FALSE
END IF
```

```
    IF valid = TRUE
        DISPLAY "Password meets the classroom rules."
    ELSE
        DISPLAY "Password does not meet the rules."
    END IF
END
```

**7. Python source code**

```
password = input("Enter a sample password: ")

has_upper = False
has_lower = False
has_digit = False

for ch in password:
    if ch.isupper():
        has_upper = True
    if ch.islower():
        has_lower = True
    if ch.isdigit():
        has_digit = True

valid = True

if len(password) < 8:
    print("Failed: must contain at least 8 characters.")
    valid = False

if not has_upper:
    print("Failed: add an uppercase letter.")
    valid = False
```

```
if not has_lower:
    print("Failed: add a lowercase letter.")
    valid = False

if not has_digit:
    print("Failed: add a digit.")
    valid = False

if valid:
    print("Password meets the classroom rules.")
else:
    print("Password does not meet the rules.")
```

**8. Test cases**

| Test case | Sample password | Expected result |
|---|---|---|
| 1 | Hello123A | Meets all classroom rules |
| 2 | hello1234 | Fails: missing uppercase letter |
| 3 | HELLO123 | Fails: missing lowercase letter |
| 4 | HelloWorld | Fails: missing digit |
| 5 | Ab1 | Fails: length is less than 8 characters |
| 6 | Abcdefg1 | Meets all classroom rules |

**9. Sample output screenshots**

These are sample console output previews, not actual screenshots captured from a Python interpreter.

Test 1 — Valid password

```
Enter a sample password: Hello123A
Password meets the classroom rules.
```

Copy sample output

Test 2 — Missing uppercase letter

```
Enter a sample password: hello1234
Failed: add an uppercase letter.
Password does not meet the rules.
```

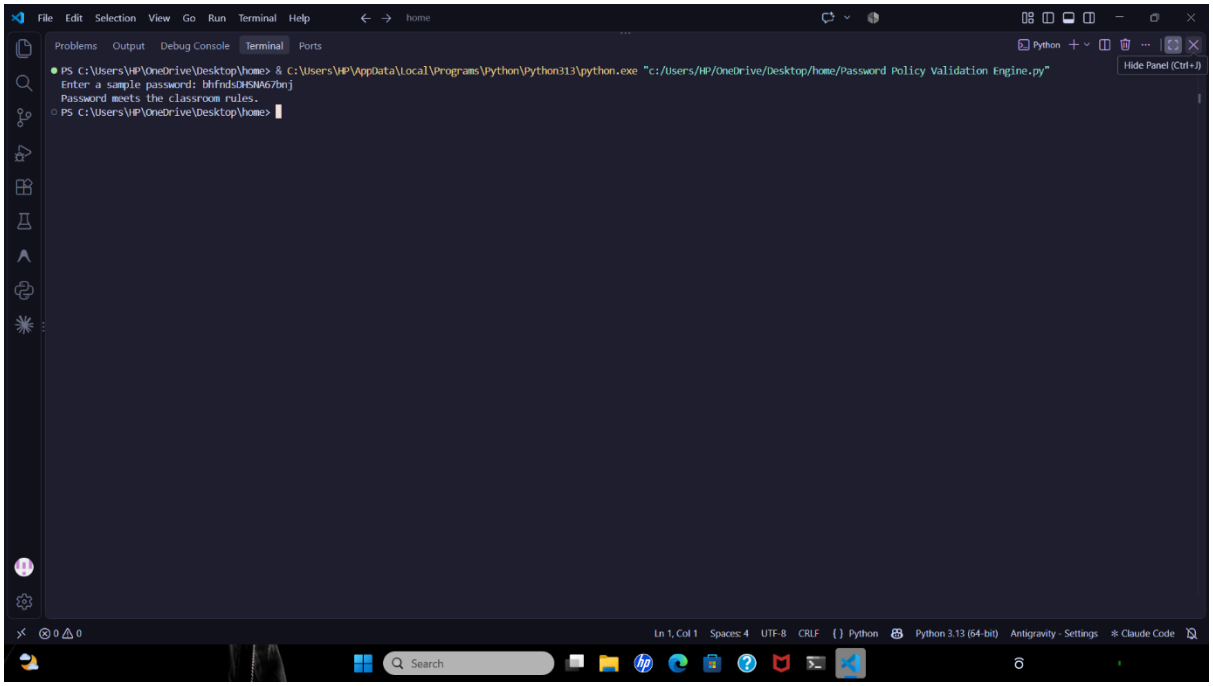
Copy sample output

Test 3 — Password too short

Enter a sample password: Ab1
Failed: must contain at least 8 characters.
Password does not meet the rules.

Copy sample output

To obtain actual screenshots for your activity record, run the program in Python IDLE or VS Code and capture the console output.




## 10. Conclusion

The Password Policy Validation Engine was successfully developed using Python. It checks the password length and verifies the presence of uppercase letters, lowercase letters, and digits. It displays appropriate error messages when the password fails any requirement. This program helps in understanding loops, conditional statements, Boolean variables, string methods, and input/output operations in Python.